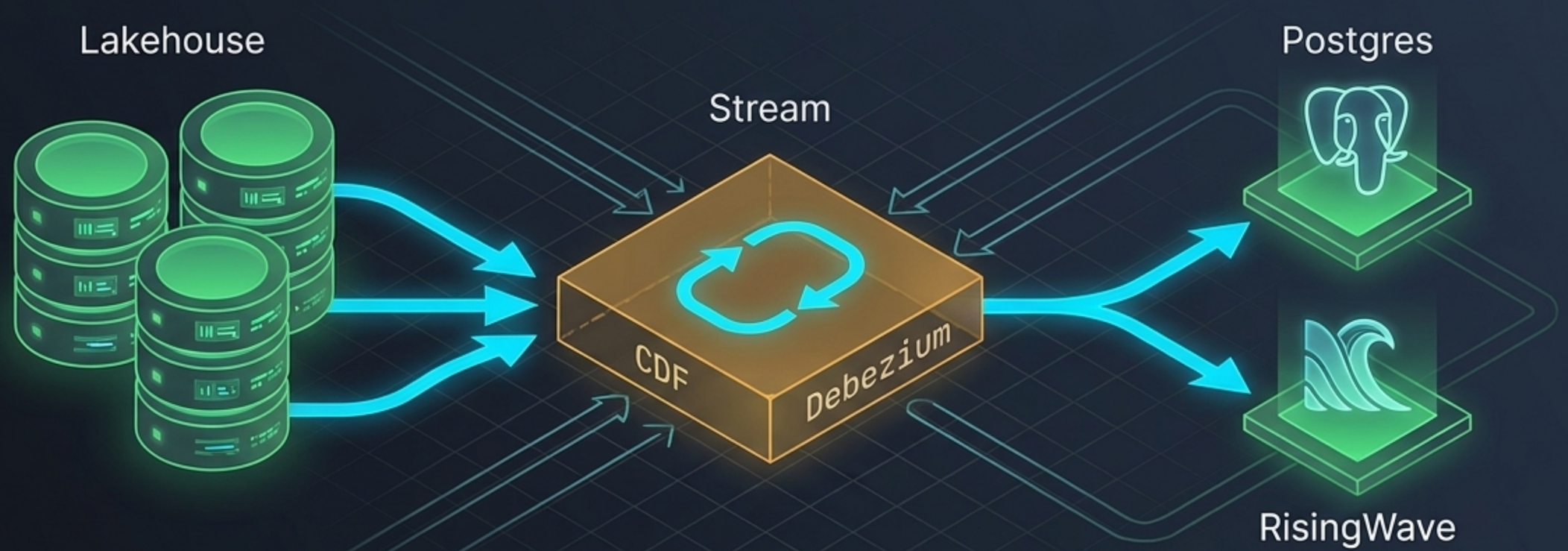


Architecting a Zero-DBU, CDC-Native Reverse ETL Pipeline

How we bypassed Databricks compute to stream Change Data Feed (CDF) events to downstream operational systems at scale.

TARGET_AUDIENCE: [Data Eng, Data Arch, Tech Lead] // CLEARANCE: Open

Gemini Notebook



TL;DR Execution Summary

Achieved Zero-DBU extraction bypassing Databricks compute via OSS Spark direct-to-storage reads.

Built native Debezium event streaming directly from Change Data Feed (CDF).

Delivered Multi-target syncs (Postgres + RisingWave) from a single Kafka topic, orchestrated by Dagster.

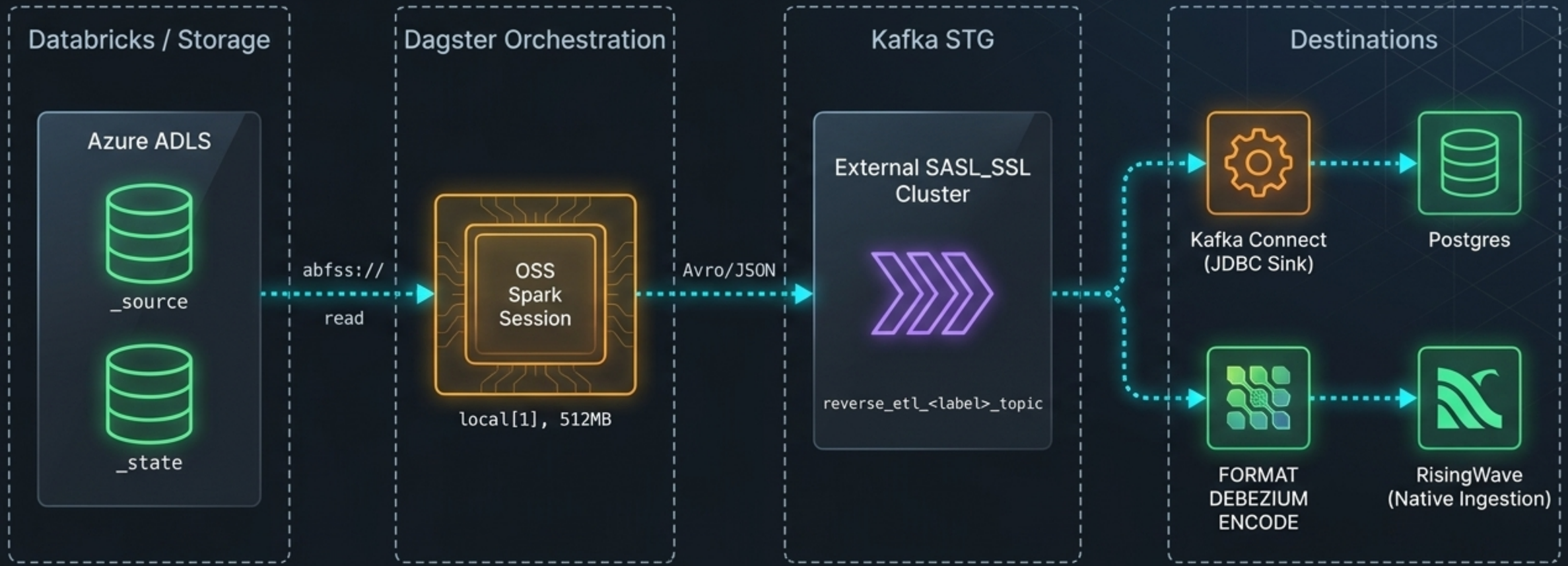
Gemini Notebook

Evaluation Criteria	drt (v1.0.0)	dlt	Custom Dagster Component
Destination Support	❌ Direct to DB only (No Kafka)	❌ Rejects Kafka destinations	✅ Writes to any Kafka consumer
Extraction Method	❌ Mirror mode (High DBU cost)	❌ Lacks CDF source	✅ Delta Change Data Feed (Pure delta)
Data Semantics	❌ Tracking tables (Loses explicit deletes)	❌ Row-level upserts	✅ Native Debezium CDC envelopes (c/u/d)

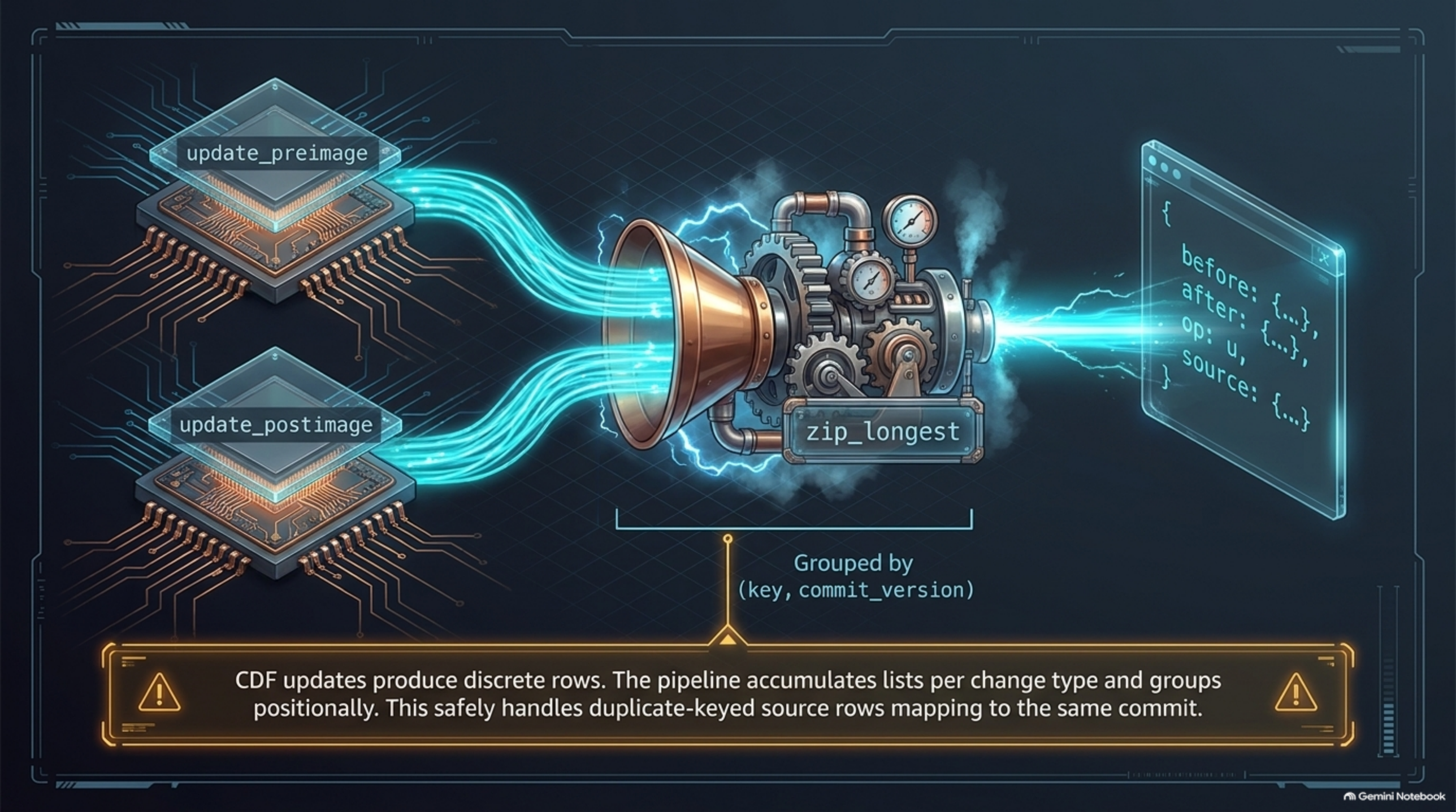
Custom build required for CDC-native extraction, Kafka delivery, and zero-DBU scalability.

Gemini Notebook

End-to-end architecture from Delta storage to downstream consumers



Gemini Notebook



Gemini Notebook

Avro encoding shrinks message payloads by over 90 percent



Requirement:	Constraint:
Shared staging Apicurio registry.	RisingWave requires temporal workaround (timestamp ingested as VARCHAR, generated as TIMESTAMPTZ).

Gemini Notebook



One Topic, Multiple Readers. Each consumer tracks its own offsets independently.

Gemini Notebook

1

Databricks

`ALTER TABLE... ADD COLUMN`

2

Dagster Asset

Reads live Delta schema

3

Dagster Asset

Registers new schema version to Apicurio Registry

4

Dagster Asset

Executes ALTER TABLE on RisingWave

5

Dagster Asset

Produces Kafka events

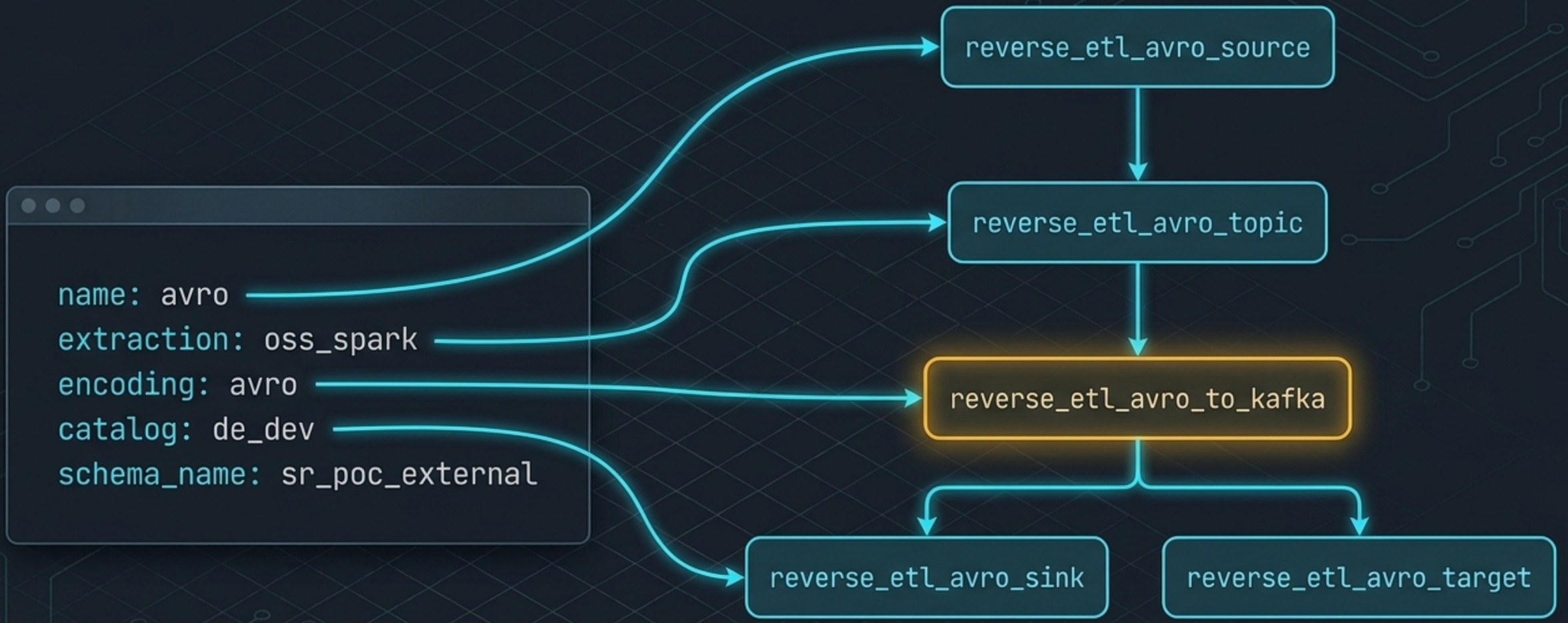
6

Postgres Sink

Auto-adds column via schema.evolution=basic

Crucial Constraint: `RisingWave` must be altered before the first event carrying the new column is produced, otherwise ingested rows default to NULL.

Gemini Notebook



The ReverseEtlSyncConfig object dynamically generates assets. A single label ensures zero name collisions, allowing isolated parallel syncs.

Gemini Notebook



Setup

Idempotent `IF NOT EXISTS` creation of tables and topics.



Sync

Manual or notebook trigger sharing a single Delta watermark. One trigger per change.



Check

The `sink_healthy` scheduled asset check. Prevents silent lag by polling the Connect REST API every 5 minutes.



Reset

Destructive teardown. Safely isolated via strict `reset_name_prefixes` allowlists.

Gemini Notebook

Databricks SQL Path (Baseline)



`table_changes()`

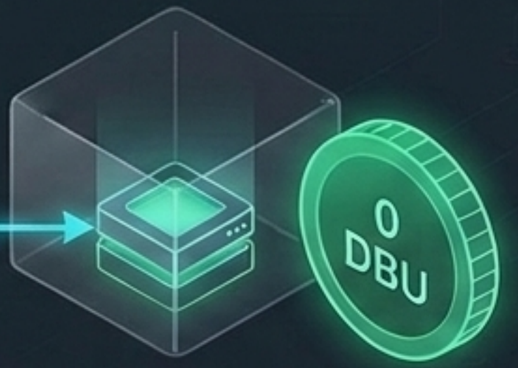
Databricks SQL Warehouse

OSS Spark Path (Zero-DBU)



Storage

`abfss://` `read`



local[1] Spark node

The State Table Decoupling



Source Table
UniForm / Iceberg enabled

Crucial Fix: OSS Spark cannot commit to UniForm tables due to Iceberg conversion hooks. The source table can be Delta or Uniform, but the `_state` table is decoupled as Plain Delta to accept OSS Spark `MERGE` commits, bringing pipeline compute costs to zero.



State/Watermark Table
Plain Delta

Gemini Notebook



Surrogate Identity Columns Required.

Delta Row Tracking is not exposed in the CDF. Syncs require an explicitly generated `rid` identity column from table creation to process deterministic upserts and deletes.



Comprehensive Schema Evolution.

The latest version now supports all types of schema evolution, including dropping columns and widening types. Changes are handled seamlessly without breaking downstream RisingWave ingestion or requiring special opt-ins.



Timestamp Zone Formatting.

Collecting `TimestampType` triggers local timezone shifts in Spark. Timestamps are formatted purely as `io.debezium.time.ZonedTimestamp` strings with 6 fractional digits to preserve UTC fidelity across systems.

Gemini Notebook